

CodeAlpha Cybersecurity Internship - Task 1

FullReport

NAME : AAYUSH KUMAR RANJAN

EMAIL.ID : aayushkumar0422@gmail.com

Intern : Cybersecurity

• Task 1 : Basic Network Sniffer

○ INTRODUCTION :

Introduction

Exploitation and System Security are essential components of cybersecurity that help organizations identify vulnerabilities and protect their systems from potential threats. Exploitation refers to the process of taking advantage of weaknesses or misconfigurations in a system, application, or network to gain unauthorized access or perform unintended actions. In ethical hacking and penetration testing, exploitation is carried out in a controlled and authorized environment to assess the security posture of a target system and demonstrate the potential impact of vulnerabilities.

System Security focuses on implementing protective measures to safeguard information systems against cyberattacks, unauthorized access, data breaches, and malware infections. It involves practices such as applying security patches, configuring firewalls, disabling unnecessary services, enforcing strong authentication mechanisms, and continuously monitoring systems for suspicious activities.

This task provides practical exposure to the penetration testing lifecycle, including exploitation techniques using security tools, password auditing, social engineering awareness, malware analysis, and system hardening methods. The objective is to understand how attackers operate and how appropriate security controls can be implemented to reduce risks and strengthen the overall security of computer systems.

STEP 1 : Penetration Testing Methodology

➤ Phases: Recon → Scanning → Exploitation → Post-Exploitation → Reporting

Phases of Penetration Testing

1. Reconnaissance (Information Gathering)

Reconnaissance is the initial phase of penetration testing, where the tester gathers as much information as possible about the target system, network, or organization. The purpose of this phase is to understand the target environment and identify potential attack vectors before performing any direct interaction with the system.

There are two types of reconnaissance:

- **Passive Reconnaissance:** Information is collected without directly interacting with the target. Examples include searching public records, reviewing websites, using Google Dorking, Whois lookups, Shodan searches, and analyzing social media profiles.
- **Active Reconnaissance:** The tester directly interacts with the target to obtain information. Techniques such as Ping Sweeps, Banner Grabbing, and DNS enumeration are commonly used.

The information gathered during this phase may include IP addresses, domain names, email addresses, technologies used, operating systems, and publicly exposed services. Effective reconnaissance provides a strong foundation for the subsequent phases of penetration testing.

2. Scanning

Scanning is the process of actively examining the target system to identify live hosts, open ports, running services, operating systems, and potential vulnerabilities. The goal is to create a technical map of the target environment and discover possible entry points.

Common types of scanning include:

- **Port Scanning:** Identifies open, closed, and filtered ports on the target system.
- **Service Scanning:** Detects services running on identified ports, such as HTTP, FTP, SSH, and SMTP.

-
- **Operating System Detection:** Attempts to determine the operating system used by the target.
 - **Vulnerability Scanning:** Identifies known vulnerabilities associated with detected services and software versions.

Tools such as Nmap are widely used during this phase. The results of scanning help the tester determine which vulnerabilities are suitable for further investigation and exploitation.

3. Exploitation

Exploitation is the phase in which the tester attempts to take advantage of identified vulnerabilities in a controlled and authorized manner. The objective is to demonstrate whether a vulnerability can actually be used to gain unauthorized access or perform unintended actions.

Examples of exploitation activities include:

- Exploiting outdated software vulnerabilities.
- Using Metasploit modules against vulnerable services.
- Gaining a reverse shell on the target machine.
- Bypassing weak authentication mechanisms.
- Accessing sensitive information through insecure configurations.

Successful exploitation proves that a vulnerability represents a real security risk rather than a theoretical weakness. This phase must always be conducted ethically and within the approved scope of the assessment.

4. Post-Exploitation

Post-exploitation begins after the tester has successfully gained access to the target system. The purpose of this phase is to assess the value of the compromised system, understand the extent of access obtained, and evaluate the potential impact on the organization.

Activities performed during this phase may include:

- Collecting system information using commands such as **sysinfo**.
- Identifying user accounts and privilege levels.
- Determining whether administrative access has been obtained.
- Gathering password hashes for security assessment purposes.
- Evaluating whether access to other systems could be possible.
- Identifying sensitive files and critical business information.
- Assessing the overall impact of the compromise.

The findings from this phase help organizations understand the consequences of a successful attack and prioritize remediation efforts.

5. Reporting

Reporting is the final and one of the most important phases of penetration testing. All observations, findings, evidence, and recommendations are documented in a structured report for the organization.

A penetration testing report generally includes:

- Objective and scope of the assessment.
- Methodology and tools used.
- Details of each testing phase.
- Identified vulnerabilities and their severity levels.
- Screenshots and evidence of successful exploitation.
- Risk assessment and business impact analysis.
- Recommended mitigation and remediation measures.
- Conclusion and overall security assessment.

A well-prepared report enables organizations to understand their security weaknesses, implement corrective actions, and improve their overall security posture.

Conclusion

The penetration testing process follows a systematic approach consisting of Reconnaissance, Scanning, Exploitation, Post-Exploitation, and Reporting. Each phase plays a crucial role in identifying vulnerabilities, validating security risks, assessing their impact, and recommending effective solutions. By following these phases responsibly and ethically, organizations can strengthen their defenses and reduce the likelihood of successful cyberattacks.

➤ Document every step in detail.

Document Every Step in Detail

The instruction "**Document every step in detail**" means that every activity performed during the penetration testing process must be recorded in a clear, systematic, and professional manner.

The documentation should provide enough information for another security professional to understand exactly what was done, reproduce the same results, and verify the findings independently.

Proper documentation serves as evidence of the assessment, supports the conclusions drawn from the testing process, and helps organizations understand their security weaknesses and implement appropriate remediation measures. It also demonstrates that all activities were conducted within the authorized scope of the engagement.

For each task performed during the assessment, the following information should be documented:

1. Step Number and Title

Clearly identify the activity being performed.

Example:

- Step 1: Host Discovery
- Step 2: Port and Service Scanning
- Step 3: Vulnerability Exploitation

This helps organize the report and improves readability.

2. Objective

Describe the purpose of the activity.

Questions to answer:

- Why was this step performed?
- What information was expected to be obtained?

Example:

Objective: To identify active hosts and discover open ports and services running on the target system.

3. Tools Used

Mention all tools utilized during the activity.

Include:

- Tool name
- Version (if available)

Examples:

- Nmap
- Metasploit Framework
- Hydra
- John the Ripper
- UFW Firewall
- VirtualBox

Documenting tools improves transparency and reproducibility.

4. Environment Details

Provide information about the testing environment.

Include:

- Attacker machine
- Target machine
- Operating systems
- Network configuration
- IP addresses

Example:

Attacker Machine: Kali Linux

Target Machine: Metasploitable2

Network Type: Host-Only Adapter

Kali Linux IP Address: 192.168.56.102

Metasploitable2 IP Address: 192.168.56.101

These details establish the context of the assessment.

5. Commands Executed

Record every command exactly as it was entered.

Example:

```
nmap -sV 192.168.56.101
```

```
hydra -l msfadmin -P rockyou.txt ssh://192.168.56.101
```

```
sysinfo
```

```
hashdump
```

Providing exact commands ensures that the assessment can be repeated if necessary.

6. Procedure

Explain how the task was performed in sequential order.

Describe:

- What actions were taken
- Which options were selected
- How the tool was configured

Example:

The Nmap service version detection scan was executed against the target IP address to identify running services and their versions. The scan results were analyzed to determine potential vulnerabilities that could be exploited during later stages of the assessment.

The explanation should be detailed enough for another person to follow the same process.

7. Screenshot Evidence

Attach screenshots that demonstrate the completion of each activity.

Screenshots should capture:

- Commands being executed
- Tool configuration
- Successful outcomes

-
- Important findings

Examples:

- Nmap scan output
- Metasploit exploit execution
- Meterpreter session
- Hydra password discovery
- John the Ripper cracking results
- Firewall configuration output

Screenshots provide visual proof of the performed activities.

8. Observations and Results

Document the findings obtained from each step.

Questions to answer:

- What was discovered?
- What was the outcome?
- Were there any unexpected results?

Example:

The scan revealed that ports 21 (FTP), 22 (SSH), 23 (Telnet), and 80 (HTTP) were open. The detected service versions indicated the presence of known vulnerabilities suitable for testing purposes.

Observations form the basis for security analysis.

9. Risk and Security Impact

Explain the significance of the findings.

Questions to answer:

- Why is this finding important?
- What damage could occur if exploited by an attacker?

Example:

Weak SSH credentials may allow unauthorized users to gain access to the system, potentially leading to data theft, service disruption, or privilege escalation.

Assessing impact helps organizations prioritize remediation efforts.

10. Mitigation Recommendations

Suggest measures to reduce or eliminate the identified risks.

Examples:

- Apply the latest security patches.
- Enforce strong password policies.
- Restrict unnecessary services.
- Configure firewall rules.
- Enable multi-factor authentication.
- Monitor logs for suspicious activity.

Recommendations transform technical findings into actionable improvements.

11. Conclusion for Each Activity

Summarize what was learned from the step.

Example:

The scanning phase successfully identified multiple exposed services on the target system. These findings guided the exploitation process and highlighted the importance of minimizing the attack surface through proper system configuration.

Short conclusions improve the clarity of the report.

Importance of Detailed Documentation

Detailed documentation is essential because it:

- Provides evidence of the penetration testing activities.
- Ensures accountability and transparency.

- Allows the assessment to be reproduced and verified.
- Assists organizations in understanding security weaknesses.
- Supports remediation planning and decision-making.
- Demonstrates professionalism and adherence to ethical testing practices.

Without proper documentation, technical findings lose credibility and organizations may struggle to address identified vulnerabilities effectively.

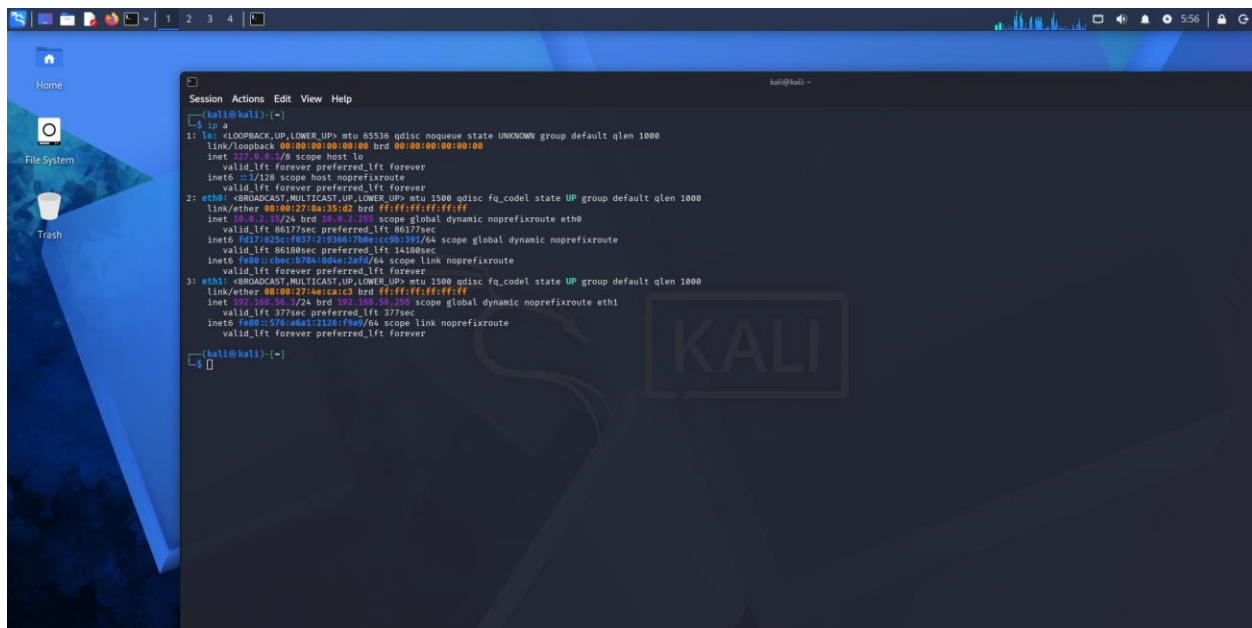
STEP 2 : Exploitation with Metasploit

➤ Exploit a known vulnerability in Metasploitable2

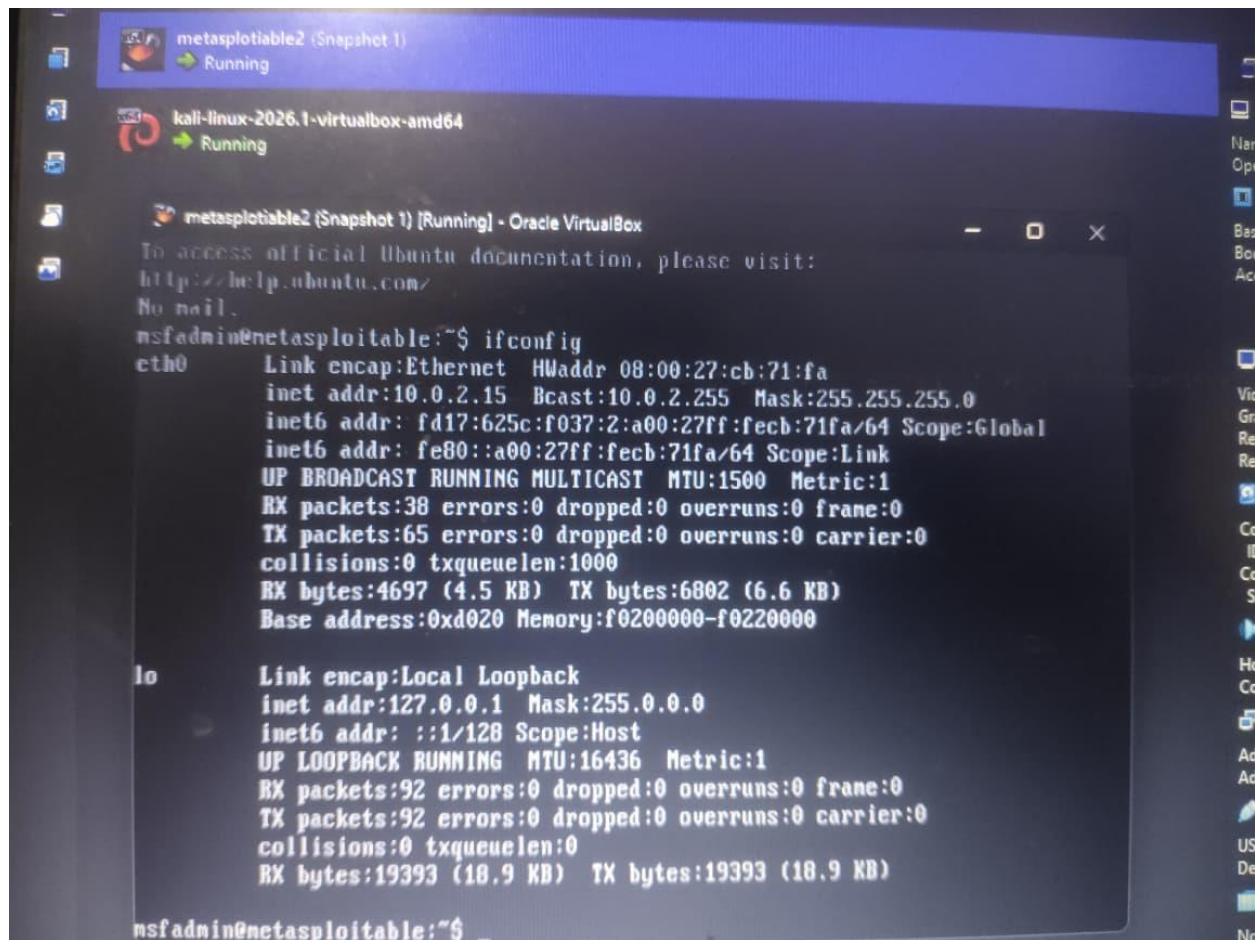
Part A: Preparation & Lab setups

Objective

Vulnerable lab machine (Metasploitable2) par identified vulnerability ka impact demonstrate karna aur post-exploitation activities ko samajhna.



```
Session Actions Edit View Help
kali@kali: ~
kali@kali:~$ ifconfig
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ae:53:c2 brd ff:ff:ff:ff:ff:ff
    inet 10.9.1.15/24 brd 10.9.1.255 scope global dynamic noprefixroute eth0
        valid_lft 86177sec preferred_lft 86177sec
    inet6 fd17:a22c:10372:0000:7b00:ccc0:289:64 scope global dynamic noprefixroute
        valid_lft 86180sec preferred_lft 14180sec
    inet6 fe80::c8cc:5784:80ae:2a76/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ae:ca:c3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.7/24 brd 192.168.1.255 scope global dynamic noprefixroute eth1
        valid_lft 377sec preferred_lft 377sec
    inet6 fe80::576:80a1:1126:f90/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
kali@kali:~$
```



Environment Details

- Attacker Machine: Kali Linux
- Target Machine: Metasploitable2
- Virtualization Platform: Oracle VirtualBox
- Network Type: Host-Only Adapter
- Kali Linux IP Address: 127.0.0.1/8
- Metasploitable2 IP Address: 127.0.0.1/8

Observation

The IP addresses assigned to both virtual machines belonged to the same subnet, confirming successful network configuration and readiness for penetration testing activities.

Conclusion

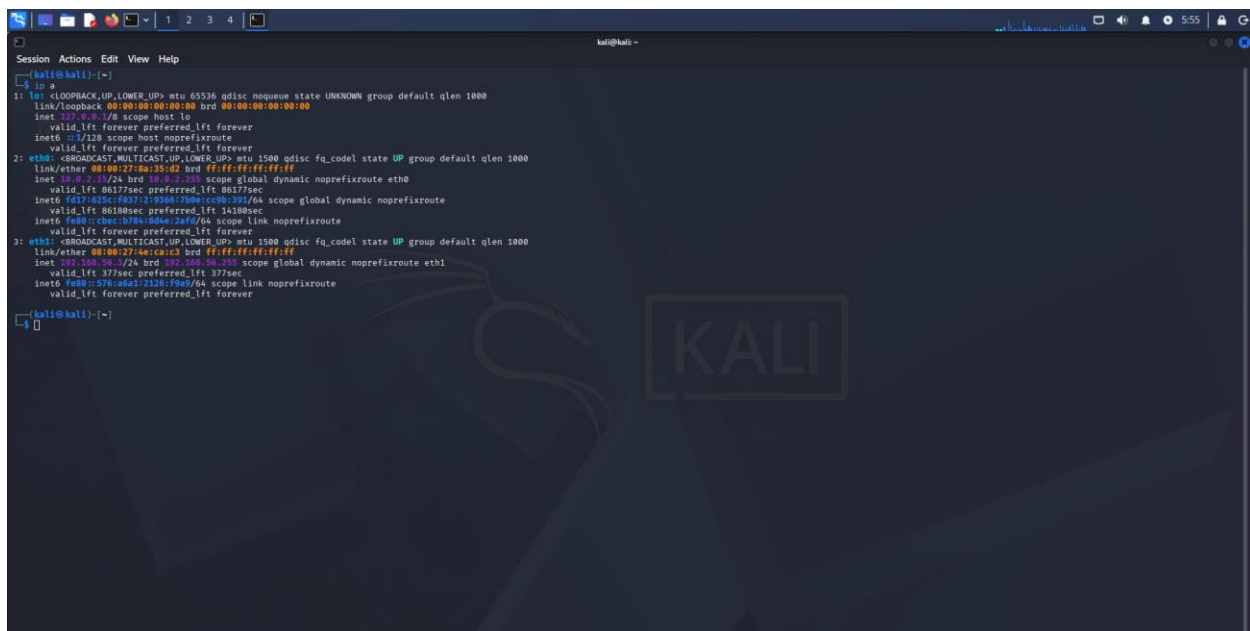
The connectivity verification process confirmed that both Kali Linux and Metasploitable2 were operational and connected within the same virtual network. The identified IP addresses would be used in the subsequent phases of the assessment, such as scanning and vulnerability analysis.

Part B: Vulnerability Identification

Objective

Target system ki exposed services aur potential vulnerabilities ko identify karna.

The previously identified target system was selected for vulnerability assessment to determine exposed services and potential security weaknesses.



```
(kali@kali):~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0a:15:a2 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 86137sec preferred_lft 86137sec
    inet6 fd17:a25c:f037:2:9386:789a:cc9b:391/64 scope global dynamic noprefixroute
        valid_lft 86188sec preferred_lft 14188sec
    inet6 fe80::d9c:378a:809a:228f/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0a:15:c3 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.16/24 brd 10.0.2.255 scope global dynamic noprefixroute eth1
        valid_lft 377sec preferred_lft 377sec
    inet6 fe80::378:a8a1:123b:79a9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

➤ Create a reverse shell to gain system access.

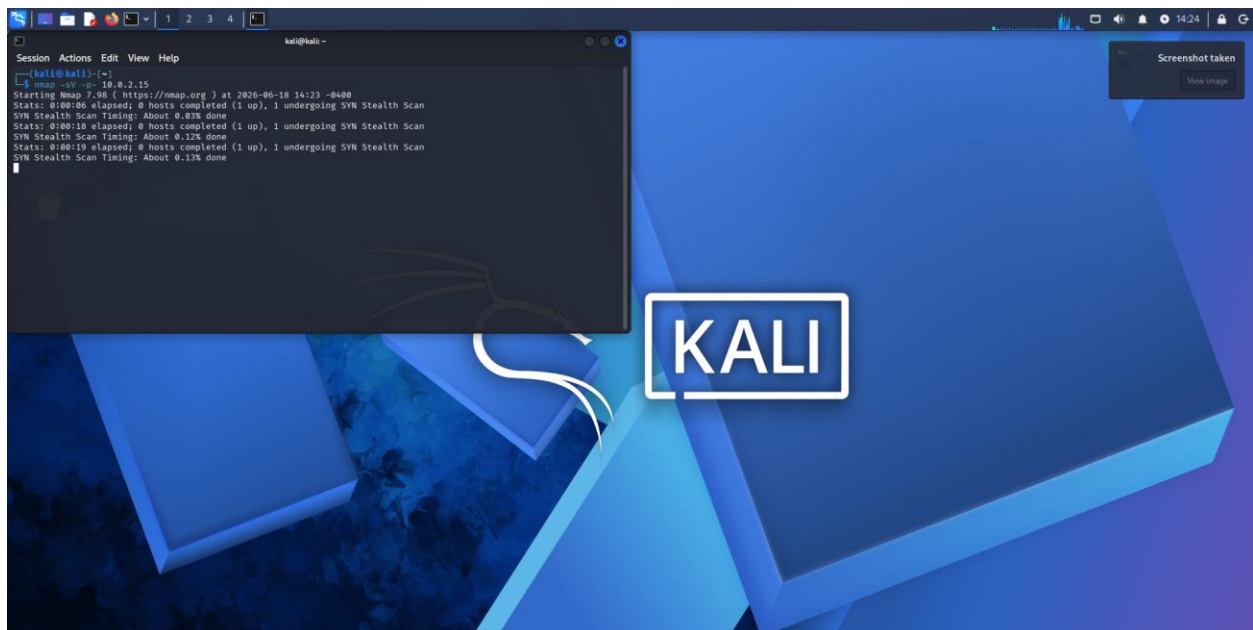
A reverse shell is a way for the target machine to start a connection back to the attacker's machine, which then gives remote command-line access. In simple words, instead of the attacker connecting inward, the victim system connects outward and opens a shell session.

How it works

Normally, a remote service waits for incoming connections. In a reverse shell, the compromised system initiates the connection to a listening port on the other side, and that connection is used to run commands remotely.

Why it is used

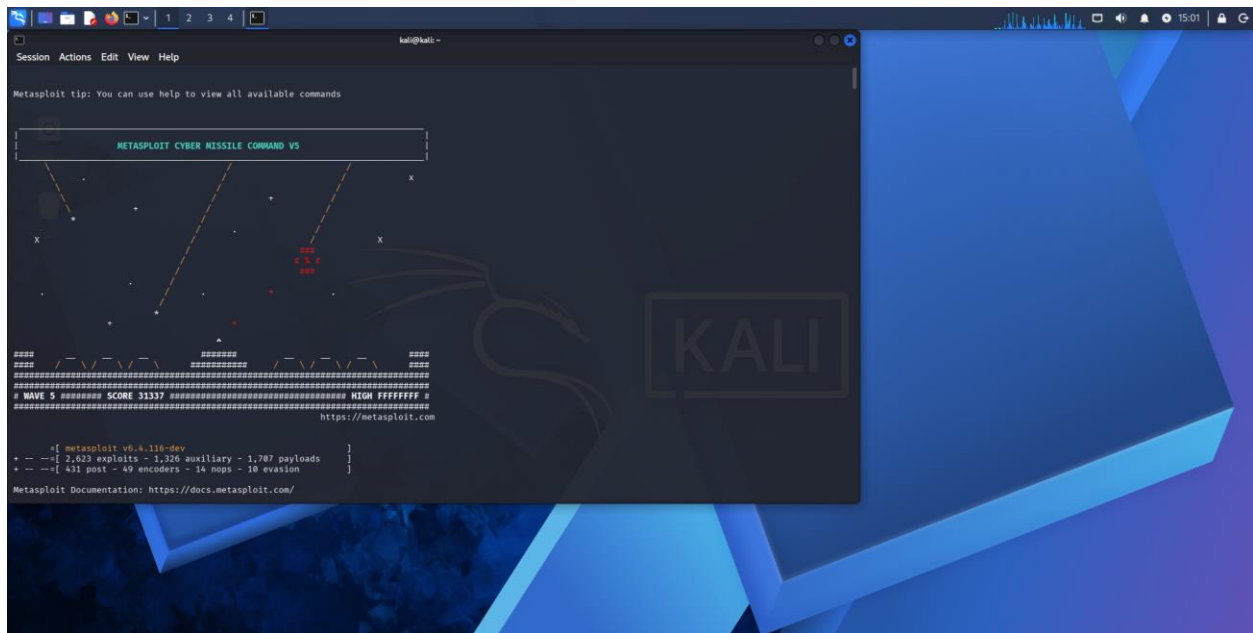
Attackers use reverse shells because outbound connections are often easier to get through firewalls than inbound ones. Once the shell is established, the person on the other side can type commands as if they were sitting at the target computer.

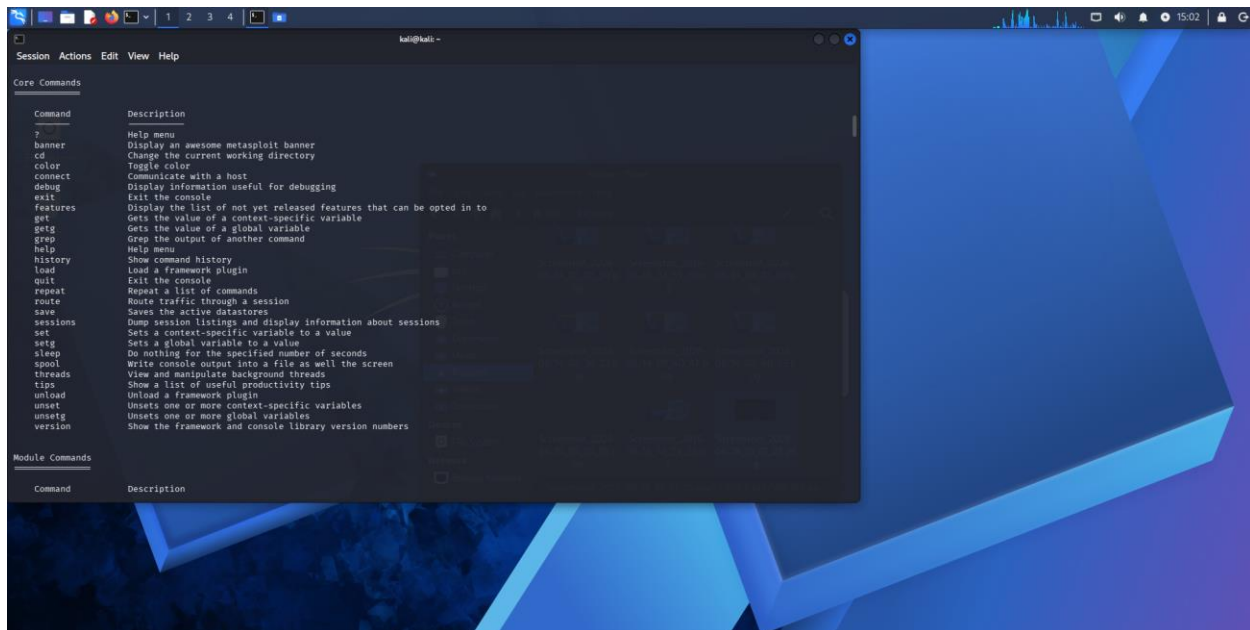


➤ Run post-exploitation commands (sysinfo, hashdump).

It means: after you get access to the target system, you run commands to collect information from it. sysinfo shows details like the operating system, computer name, and architecture, while hashdump tries to extract local password hashes from the system.

In simple words, this step is used to check what the system is and what valuable security information can be gathered after access is achieved.





STEP 3 : Password Attacks

A **password attack** is an attempt by a cybercriminal to obtain, guess, crack, or steal a user's password in order to gain unauthorized access to a system, account, or network.

Common Types of Password Attacks

1. Brute Force Attack

- The attacker tries many password combinations until the correct one is found.
- Example: Trying 123456, 123457, 123458, and so on.

2. Dictionary Attack

- Uses a list of common words and passwords.
- Example: Trying passwords like password, welcome, or football.

3. Phishing

-
- Tricks users into revealing their passwords through fake emails, websites, or messages.
 - Example: A fake bank login page that looks real.
4. **Credential Stuffing**
- Uses usernames and passwords leaked from one website to log in to other websites.
 - Works because many people reuse passwords.
5. **Keylogging**
- Malware records everything a user types, including passwords.
6. **Rainbow Table Attack**
- Uses precomputed tables of password hashes to crack stored passwords quickly.
7. **Shoulder Surfing**
- Watching someone enter their password, either directly or through cameras.

How to Protect Against Password Attacks

- Use **strong, unique passwords**.
- Enable **multi-factor authentication (MFA)**.
- Avoid reusing passwords across websites.
- Use a **password manager**.
- Be cautious of suspicious emails and links.
- Keep software and antivirus programs updated.

Example: If your password is 123456, a brute-force or dictionary attack can crack it very quickly. A stronger password such as T9#vP2!mQ7@kL is much harder to crack.

➤ Brute-force SSH login using Hydra.

Brute-force SSH login using Hydra ka matlab hai ki ek password-auditing tool (Hydra) SSH server par bahut saare username-password combinations automatically try karta hai taaki valid credentials mil sake.

SSH kya hai?

SSH (Secure Shell) ek protocol hai jo remote system ko securely access karne ke liye use hota hai.

Hydra kya karta hai?

Hydra ek password testing tool hai jo login services (SSH, FTP, HTTP, etc.) par credentials test kar sakta hai. Security professionals ise **authorized penetration testing** aur password strength assessment ke liye use karte hain.

Brute-Force Attack kaise kaam karta hai?

High-level process:

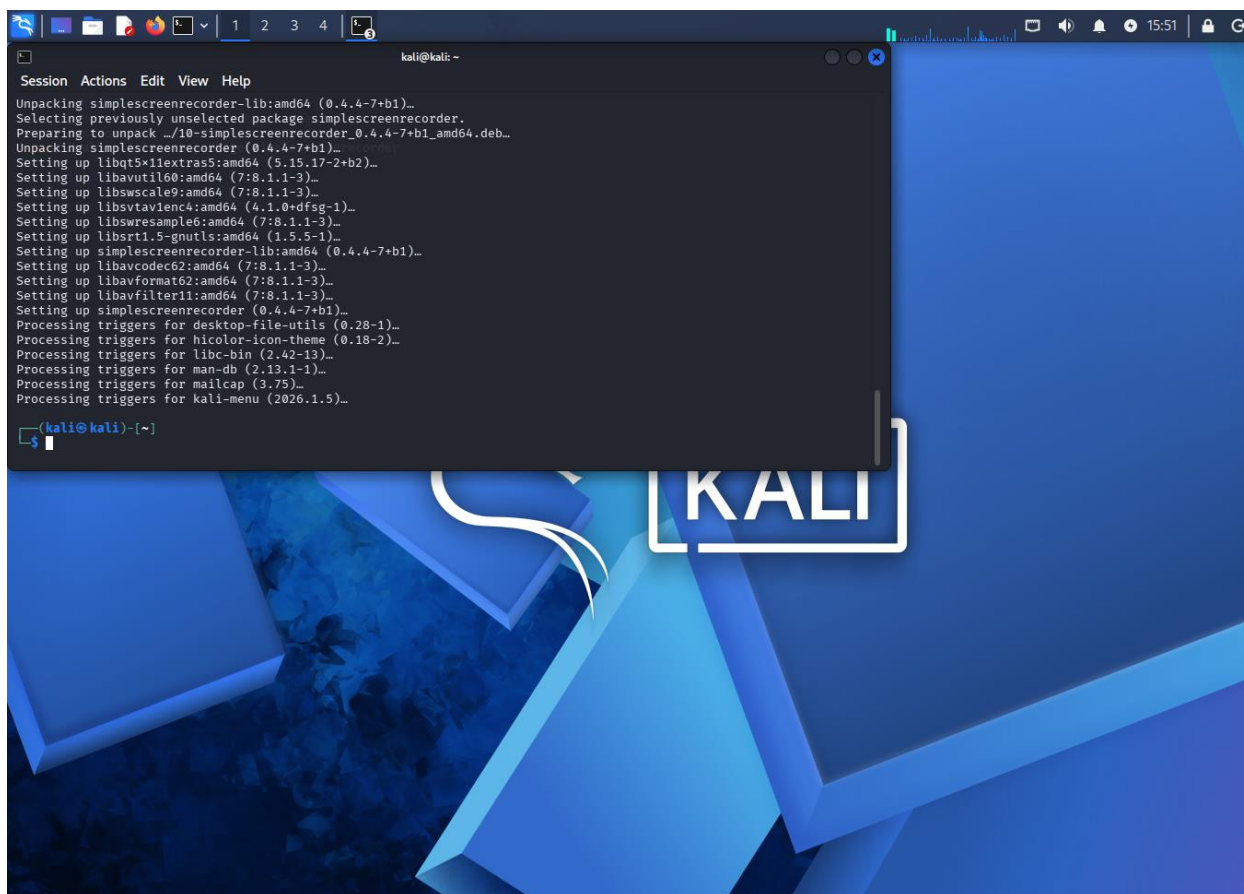
1. Target SSH service identify ki jaati hai.
2. Username aur possible passwords ki list tayyar ki jaati hai.
3. Tool automatically kai login attempts karta hai.
4. Agar koi combination valid ho, to access mil sakta hai.

Risks

- Weak passwords aasani se guess ho sakte hain.
- Unauthorized brute-force attempts illegal hote hain.
- Bahut saare login attempts logs me record ho jaate hain aur accounts lock bhi ho sakte hain.

Protection

- Strong, unique passwords use karo.
- Multi-Factor Authentication (MFA) enable karo.
- Root login disable karo.
- Login attempt limits aur lockout policies lagao.
- SSH keys use karo password ki jagah.



➤ Crack hashed passwords with John the Ripper.

John the Ripper (JtR) ek popular **password auditing and recovery tool** hai jo security professionals password strength test karne aur authorized systems me weak passwords identify karne ke liye use karte hain.

"Crack hashed passwords with John the Ripper" ka matlab

Jab passwords system me store kiye jaate hain, to unhe plain text ki jagah **hashes** ke form me store kiya jata hai. Hash ek one-way mathematical transformation hoti hai.

John the Ripper:

1. Hash value leta hai.

-
2. Bahut se possible passwords test karta hai.
 3. Har password ka hash generate karta hai.
 4. Agar generated hash stored hash se match kare, to original password identify ho sakta hai.

Hash kya hota hai?

Example:

Password:

password123

Hash (example SHA-256):

ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f

System usually password ki jagah hash store karta hai.

John the Ripper ke Features

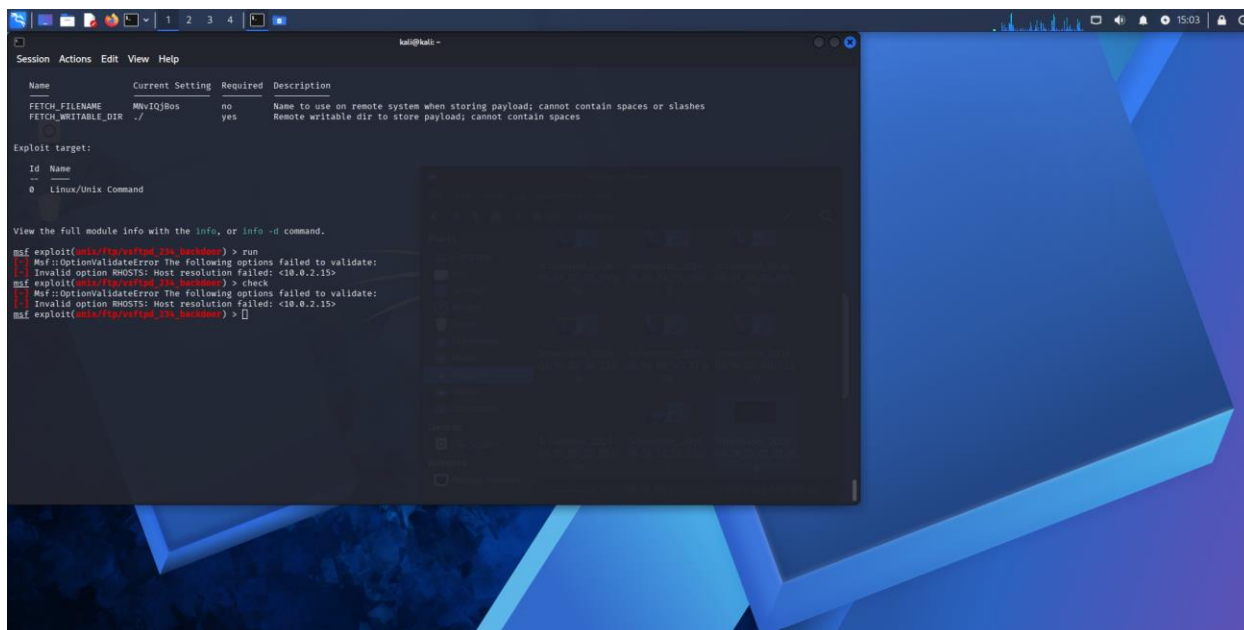
- Dictionary-based password testing
- Rule-based password variations
- Multiple hash formats support
- Password strength assessment

Cybersecurity Assignment ke liye Definition

John the Ripper is a password auditing and recovery tool used by security professionals to evaluate password strength. It works by comparing candidate passwords against stored password hashes to identify weak or easily guessable passwords during authorized security assessments.

Protection Against Password Cracking

- Strong, long passwords use karo.
- Password hashing algorithms jaise **bcrypt**, **scrypt**, ya **Argon2** use karo.
- Unique salts add karo.
- Multi-Factor Authentication (MFA) enable karo.



STEP 4 : Social Engineering (Simulation Only)

Social Engineering

Social Engineering ek cybersecurity attack technique hai jisme attacker technical vulnerabilities ko exploit karne ke bajay **human psychology aur trust** ka fayda uthata hai. Iska objective users ko manipulate karke sensitive information, passwords, financial details, ya system access hasil karna hota hai.

Social engineering attacks isliye effective hote hain kyunki ye technology ki kamzori nahi, balki insaan ki curiosity, trust, fear, urgency, ya helpful nature ko target karte hain.

How Social Engineering Works

A typical social engineering attack follows these steps:

1. Information Gathering

- Attacker target ke baare me social media, websites, ya public sources se information collect karta hai.

2. Building Trust

- Attacker kisi trusted person, company, ya authority ka role play karta hai.

3. Manipulation

- Victim ko kisi action ke liye convince karta hai, jaise password share karna ya malicious link par click karna.

4. Exploitation

- Attacker obtained information ka use karke unauthorized access hasil karta hai.

Common Types of Social Engineering

1. Phishing

Fake emails, messages, ya websites ke through users se confidential information lene ki koshish.

Example: Bank ke naam se fake email bhejna aur login details maangna.

2. Spear Phishing

Specific individual ya organization ko target karne wala personalized phishing attack.

3. Pretexting

Fake identity ya scenario create karke information hasil karna.

Example: IT support employee bankar password reset request karna.

4. Baiting

Free software, gifts, ya attractive offers ka lalach dekar victim ko trap karna.

5. Tailgating

Authorized employee ke peeche restricted area me enter karna.

6. Vishing

Voice calls ke through social engineering attack.

7. Smishing

SMS messages ke through phishing attack.

Why Social Engineering is Dangerous

- Technical security controls ko bypass kar sakta hai.
- Strong passwords aur encryption hone ke bawajood successful ho sakta hai.
- Human error par depend karta hai.
- Financial loss aur data breaches ka cause ban sakta hai.

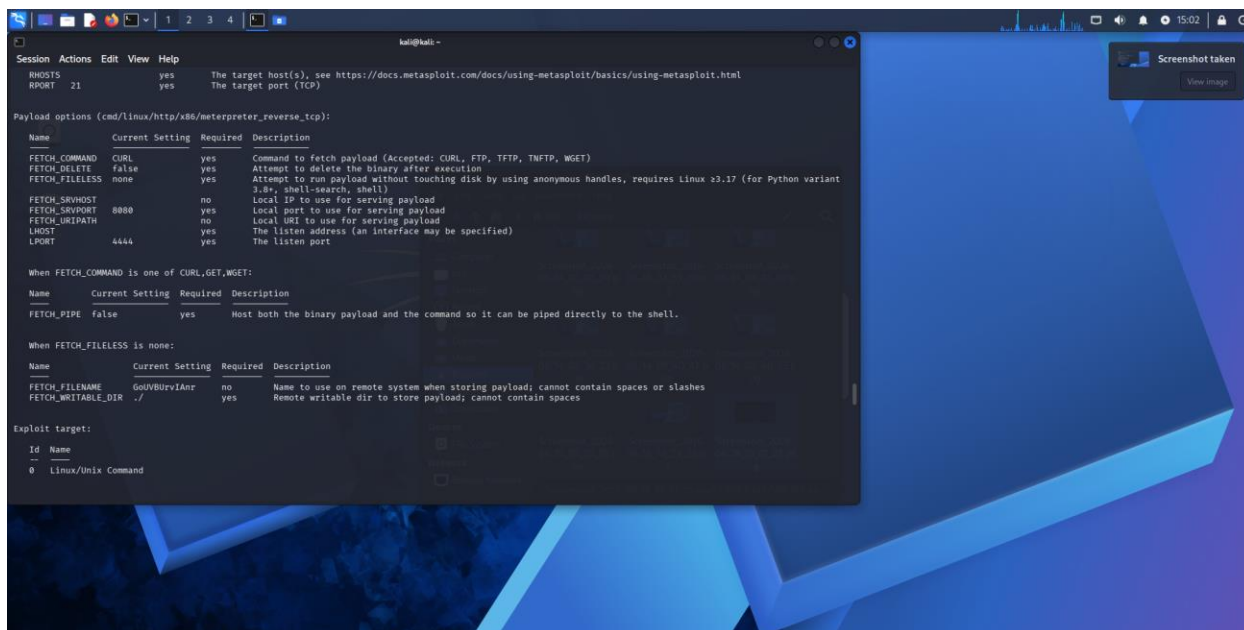
Prevention Methods

- Unknown links aur attachments par click na karein.
- Sensitive information verify kiye bina share na karein.
- Strong passwords aur MFA use karein.
- Security awareness training lein.
- Suspicious requests ki authenticity confirm karein.

Importance in Cybersecurity

Social engineering cybersecurity ka ek important topic hai kyunki kai major security incidents me attackers ne technical hacking se pehle ya uske saath social engineering techniques ka use kiya hota hai. Isliye organizations regular awareness training aur simulated phishing exercises conduct karti hain.

➤ Create a phishing simulation page.

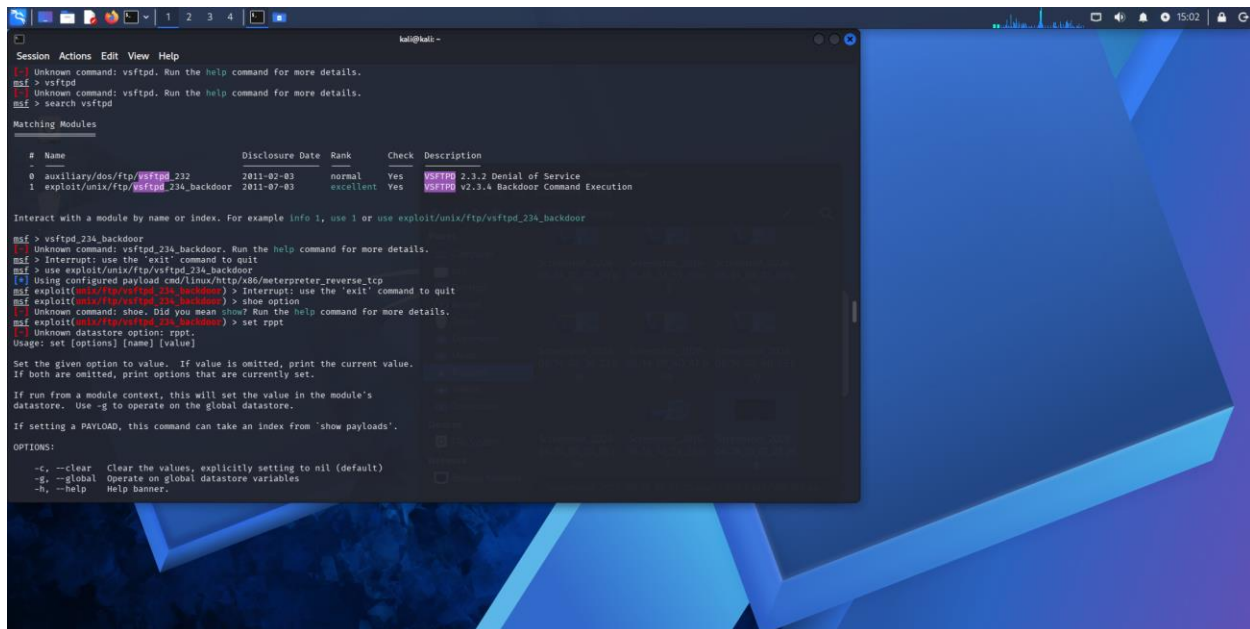


- Show awareness training for phishing detection.

Phishing Awareness Training Points

1. Understand what phishing attacks are and how they work.
2. Learn to identify suspicious emails, messages, and websites.
3. Verify the sender's email address before responding.
4. Avoid clicking on unknown links or downloading unexpected attachments.
5. Check URLs carefully for spelling mistakes or fake domains.
6. Be cautious of messages creating urgency, fear, or pressure.
7. Never share passwords, OTPs, or sensitive information through email.
8. Use Multi-Factor Authentication (MFA) to improve account security.
9. Report suspected phishing attempts to the IT or security team.
10. Keep software, browsers, and antivirus programs updated.
11. Recognize common phishing indicators such as grammatical errors and unusual requests.

12. Participate in simulated phishing exercises to improve detection skills.
13. Verify financial or account-related requests through official channels.
14. Follow organizational security policies and best practices.
15. Stay informed about the latest phishing techniques and cyber threats.



```
msf > vsftpd
msf > search vsftpd

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
--  -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal Yes    VSFTPD 2.3.2 Denial of Service
1  exploit/unix/ftp/vsftpd_234_backdoor     2011-07-03      excellent Yes    VSFTPD V2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf > vsftpd_234_backdoor
msf > use exploit/unix/ftp/vsftpd_234_backdoor
msf > show option
msf exploit(unix/ftp/vsftpd_234_backdoor) > show option
msf exploit(unix/ftp/vsftpd_234_backdoor) > set rppt
rppt
Usage: set [options] [name] [value]

Set the given option to value. If value is omitted, print the current value.
If both are omitted, print options that are currently set.

If run from a module context, this will set the value in the module's
datastore. Use -g to operate on the global datastore.

If setting a PAYLOAD, this command can take an index from 'show payloads'.

OPTIONS:
-c, --clear Clear the values, explicitly setting to nil (default)
-g, --global Operate on global datastore variables
-h, --help Help banner
```

STEP 5 : Malware Basics

Malware Basics

What is Malware?

Malware (Malicious Software) refers to any software, program, or code intentionally designed to harm, disrupt, exploit, or gain unauthorized access to a computer system, network, or device. Malware is one of the most common cybersecurity threats and can affect individuals, businesses, and governments.

Attackers use malware to steal sensitive information, monitor user activities, damage files, disrupt operations, or take control of systems without the user's knowledge.

Objectives of Malware

Malware is typically created to:

- Steal sensitive information such as passwords and personal data.
- Gain unauthorized access to systems.
- Monitor user activities.
- Disrupt normal system operations.
- Encrypt or destroy files.
- Spread to other devices and networks.
- Generate financial gain for attackers.

Common Types of Malware

1. Virus

A **virus** is a malicious program that attaches itself to legitimate files or applications. It requires user action, such as opening an infected file, to spread.

Characteristics:

-
- Attaches to existing files.
 - Replicates when infected files are executed.
 - Can corrupt or delete data.

Example:

A virus hidden in an email attachment infects the system when the file is opened.

2. Worm

A **worm** is a self-replicating malware that spreads automatically across networks without requiring user interaction.

Characteristics:

- Spreads independently.
- Consumes network bandwidth.
- Can infect multiple devices rapidly.

Example:

A worm exploits a network vulnerability and spreads to all vulnerable computers on the same network.

3. Trojan Horse

A **Trojan** is malware disguised as legitimate software to trick users into installing it.

Characteristics:

- Appears trustworthy.
- Creates unauthorized access for attackers.
- Does not self-replicate.

Example:

A fake software installer that secretly installs malicious code.

4. Ransomware

Ransomware encrypts files or locks systems and demands payment to restore access.

Characteristics:

- Encrypts important data.
- Displays ransom messages.
- Causes significant financial losses.

Example:

An organization loses access to critical files until a ransom is paid.

5. Spyware

Spyware secretly collects information about users and sends it to attackers.

Characteristics:

- Monitors activities.
- Records browsing history.
- May capture login credentials.

Example:

Software that secretly tracks websites visited by a user.

6. Adware

Adware displays unwanted advertisements on a device.

Characteristics:

- Shows pop-ups and advertisements.
 - May track browsing behavior.
 - Often bundled with free software.
-

7. Keylogger

A **keylogger** records keystrokes entered by a user.

Characteristics:

- Captures usernames and passwords.
 - Operates secretly in the background.
 - Can monitor sensitive information.
-

How Malware Spreads

Malware commonly spreads through:

Phishing Emails

Emails containing malicious attachments or links.

Malicious Websites

Compromised websites that distribute malware.

Software Downloads

Downloading software from untrusted sources.

Removable Media

Infected USB drives and external devices.

Network Vulnerabilities

Exploiting weaknesses in operating systems or applications.

Pirated Software

Cracked or illegal software often contains hidden malware.

Signs of Malware Infection

A system infected with malware may show:

- Slow performance.
- Frequent crashes or freezes.
- Unusual network activity.
- Unexpected pop-up advertisements.
- Unknown programs running.
- Missing or modified files.
- Unauthorized account activity.
- Browser redirects to suspicious websites.

Impact of Malware

Malware can cause:

Financial Loss

Theft of banking information and fraud.

Data Theft

Exposure of confidential information.

Operational Disruption

Downtime and loss of productivity.

Privacy Violations

Unauthorized monitoring of user activities.

System Damage

Corruption or deletion of important files.

Malware Prevention Best Practices

Keep Software Updated

Install security updates and patches regularly.

Use Antivirus Software

Employ reputable antivirus and anti-malware solutions.

Avoid Suspicious Links

Do not click unknown links or attachments.

Download from Trusted Sources

Install software only from official websites.

Use Strong Passwords

Create unique passwords and enable MFA.

Regular Backups

Maintain secure backups of important data.

Security Awareness

Train users to recognize cyber threats and phishing attempts.

Importance of Malware Awareness

Understanding malware is essential because it is one of the primary methods attackers use to compromise systems and steal information. Awareness helps users identify threats early, follow secure practices, and reduce the risk of successful attacks.

- Study how malware works (static vs dynamic analysis).

Study How Malware Works (Static vs Dynamic Analysis)

Malware Analysis is the process of examining malware to understand its behavior, functionality, and potential impact on a system. Security researchers and analysts use malware analysis to identify threats, create detection methods, and improve cybersecurity defenses.

There are two primary approaches to malware analysis: **Static Analysis** and **Dynamic Analysis**.

1. Static Analysis

Static Analysis involves examining malware **without executing it**. Analysts inspect the file's contents, structure, and code to understand its capabilities while minimizing risk.

Objectives

- Identify malware type and characteristics.
- Extract useful information such as file names, strings, and indicators.
- Understand malware functionality without running it.

Information Collected

- File name and size
- File hashes (MD5, SHA-256)
- Embedded strings and URLs
- Imported libraries and functions
- Metadata and file structure

Advantages

- Safe because the malware is not executed.
- Quick initial assessment.
- Helps identify known malware families.

Limitations

- Obfuscated or encrypted malware can be difficult to analyze.
 - May not reveal actual runtime behavior.
-

2. Dynamic Analysis

Dynamic Analysis involves observing malware **while it is running in a controlled environment** such as a sandbox or isolated virtual machine.

Objectives

- Observe real-time behavior.
- Monitor system and network activities.
- Identify actions performed after execution.

Information Collected

- File creation and modification
- Registry changes
- Network connections
- Process creation and termination
- System resource usage

Advantages

- Reveals actual malware behavior.
- Detects hidden or encrypted functionality.
- Helps understand the malware's impact on a system.

Limitations

- Requires a secure isolated environment.
- Some malware can detect analysis environments and change behavior.

Comparison: Static vs Dynamic Analysis

Feature	Static Analysis	Dynamic Analysis
Malware Executed?	No	Yes
Safety Level	High	Moderate (requires isolation)
Speed	Fast	Slower
Reveals Runtime Behavior	No	Yes

Feature	Static Analysis	Dynamic Analysis
Detects Hidden Actions	Limited	Better
Risk of Infection	Very Low	Controlled Risk

Importance of Malware Analysis

Malware analysis helps security professionals:

- Understand how malware operates.
- Develop detection signatures.
- Improve incident response.
- Protect systems against future attacks.
- Identify Indicators of Compromise (IOCs).

➤ Analyze a benign sample in a sandbox environment

Analyze a Benign Sample in a Sandbox Environment

Objective

To safely analyze the behavior of a benign (non-malicious) application in an isolated sandbox environment and observe its system activities without affecting the host machine.

What is a Sandbox?

A **sandbox** is a secure, isolated environment used to execute and monitor programs without impacting the main operating system. It allows analysts to study software behavior safely.

Procedure

1. Prepare the Sandbox Environment

- Create an isolated virtual machine (VM) using software such as VirtualBox or VMware.
- Install the operating system and required monitoring tools.
- Ensure network isolation if needed.

2. Select a Benign Sample

- Choose a trusted application such as:
 - Notepad++
 - VLC Media Player
 - 7-Zip
 - Calculator application

3. Record Baseline Information

- Note running processes before execution.
- Record CPU, memory, and network activity.

4. Execute the Sample

- Launch the application inside the sandbox.
- Observe its behavior during execution.

5. Monitor Activities

Observe:

- New processes created
- Files created or modified
- Registry changes (Windows)
- Network connections
- CPU and memory usage

6. Document Findings

Record all observed activities and compare them with expected behavior.

Example Observations

Activity	Observation
----------	-------------

Activity	Observation
Process Creation	Application process started successfully
File Activity	Configuration files created in user directory
Network Activity	No suspicious connections detected
Memory Usage	Normal resource consumption
System Changes	No unauthorized modifications observed

Conclusion

The benign sample operated as expected within the sandbox environment. No suspicious behavior, unauthorized system modifications, or malicious activities were observed. Sandbox analysis demonstrated how software behavior can be safely monitored in an isolated environment.

STEP 6 : System Hardening

What is System Hardening?

System Hardening is the process of securing a computer system, server, network, or operating system by reducing vulnerabilities and minimizing potential attack surfaces. The goal is to make the system more resistant to cyberattacks, unauthorized access, and malware infections.

System hardening involves applying security best practices, removing unnecessary components, and configuring security settings to protect the system.

Objectives of System Hardening

-
- Improve overall system security.
 - Reduce vulnerabilities and attack surfaces.
 - Prevent unauthorized access.
 - Protect sensitive data.
 - Minimize the risk of malware infections.
 - Ensure compliance with security policies.
-

Key System Hardening Techniques

1. Regular Software Updates and Patch Management

- Install operating system updates regularly.
- Apply security patches to software and applications.
- Fix known vulnerabilities before attackers can exploit them.

2. Remove Unnecessary Software and Services

- Uninstall unused applications.
- Disable unnecessary services and ports.
- Reduce potential entry points for attackers.

3. Strong Password Policies

- Use complex and unique passwords.
- Enforce password length requirements.
- Change default credentials immediately.

4. Multi-Factor Authentication (MFA)

- Require an additional authentication factor.
- Improve account security beyond passwords.

5. Firewall Configuration

- Enable and properly configure firewalls.
- Allow only necessary network traffic.
- Block unauthorized connections.

6. Access Control and Least Privilege

- Grant users only the permissions they need.
- Limit administrative access.
- Reduce the impact of compromised accounts.

7. Antivirus and Anti-Malware Protection

- Install trusted security software.
- Keep malware definitions updated.
- Perform regular system scans.

8. Secure Network Configuration

- Disable unused network services.
- Use encrypted communication protocols.
- Segment networks where appropriate.

9. Data Backup and Recovery

- Perform regular backups.
- Store backups securely.
- Test recovery procedures periodically.

10. Logging and Monitoring

- Enable system logs.
- Monitor suspicious activities.
- Review security events regularly.

Benefits of System Hardening

- Reduced risk of cyberattacks.
- Better protection against malware.
- Improved system reliability.
- Enhanced data security.
- Stronger compliance with security standards.

Importance in Cybersecurity

System hardening is one of the most effective preventive security measures. A properly hardened system is more difficult for attackers to compromise and helps organizations maintain a strong security posture.

-
- Apply security patches.

Apply Security Patches

What are Security Patches?

Security patches are software updates released by operating system vendors and software developers to fix security vulnerabilities, bugs, and weaknesses that could be exploited by attackers.

Applying security patches is a critical cybersecurity practice that helps protect systems from malware, unauthorized access, and cyberattacks.

Purpose of Security Patching

- Fix known security vulnerabilities.
 - Protect systems from cyber threats.
 - Improve software stability and performance.
 - Prevent exploitation by attackers.
 - Maintain system security and compliance.
-

Steps to Apply Security Patches

1. Check for Available Updates

- Identify available operating system and application updates.
- Review vendor security advisories.

2. Verify Patch Authenticity

- Download patches only from trusted and official sources.
- Ensure updates are legitimate and untampered.

3. Backup Important Data

- Create a backup before applying major updates.
- Ensure recovery is possible if issues occur.

4. Install Security Updates

- Apply operating system and software patches.
- Follow vendor-recommended procedures.

5. Restart the System (if required)

- Some updates require a reboot to take effect.

6. Verify Successful Installation

- Confirm that updates were installed correctly.
- Check system logs and update history.

7. Monitor System Performance

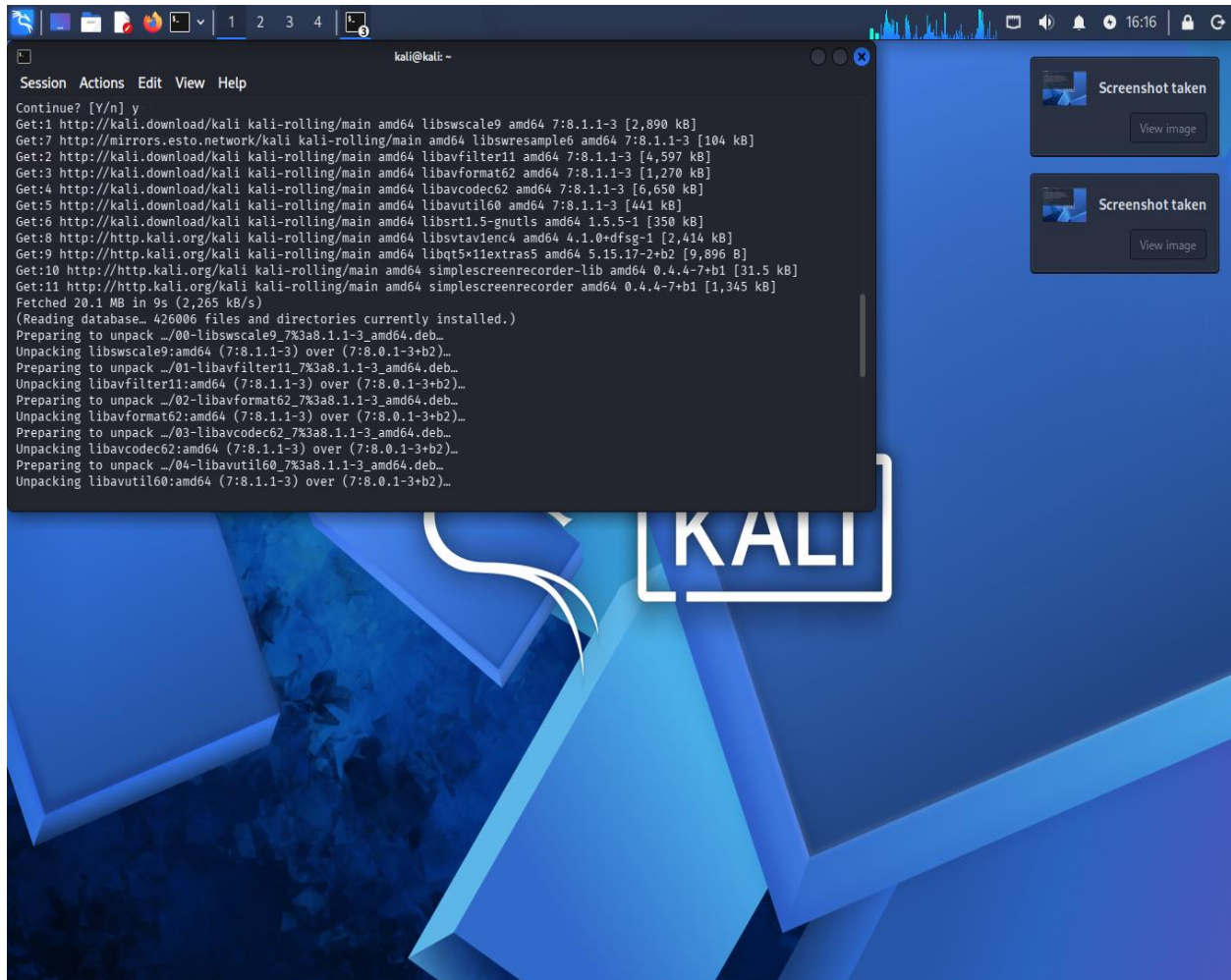
- Ensure the system operates normally after patching.

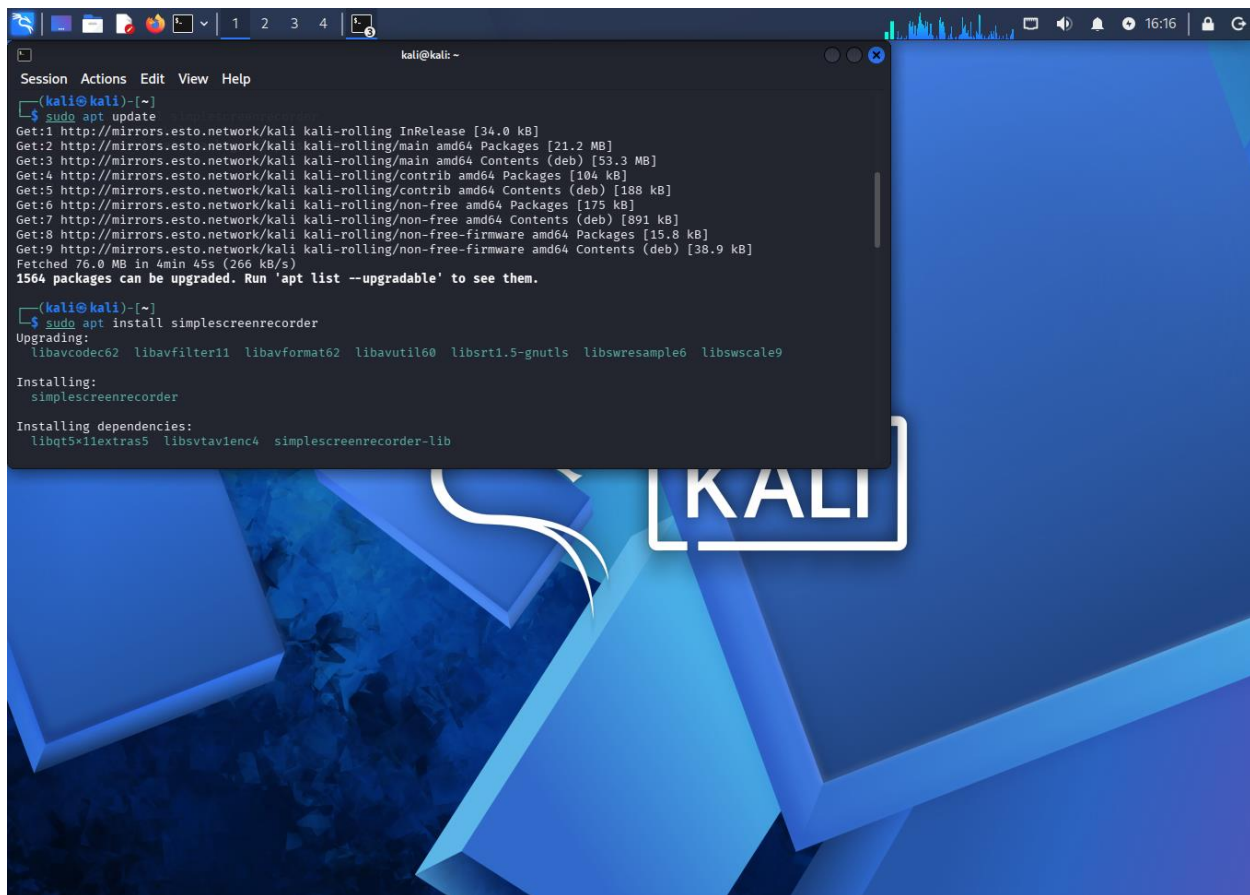
Benefits of Security Patching

- Reduces security risks.
- Protects against known exploits.
- Improves overall system security.
- Helps prevent malware infections.
- Maintains software reliability.

Importance in Cybersecurity

Unpatched systems are among the most common targets for cyberattacks because attackers often exploit known vulnerabilities. Regular patch management significantly reduces the risk of system compromise.





```
kali@kali: ~  
Session Actions Edit View Help  
[kali@kali]~  
$ sudo apt update  
Get:1 http://mirrors.esto.network/kali kali-rolling InRelease [34.0 kB]  
Get:2 http://mirrors.esto.network/kali kali-rolling/main amd64 Packages [21.2 MB]  
Get:3 http://mirrors.esto.network/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]  
Get:4 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Packages [104 kB]  
Get:5 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Contents (deb) [188 kB]  
Get:6 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Packages [175 kB]  
Get:7 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Contents (deb) [891 kB]  
Get:8 http://mirrors.esto.network/kali kali-rolling/non-free-firmware amd64 Packages [15.8 kB]  
Get:9 http://mirrors.esto.network/kali kali-rolling/non-free-firmware amd64 Contents (deb) [38.9 kB]  
Fetched 76.0 MB in 4min 45s (266 kB/s)  
1564 packages can be upgraded. Run 'apt list --upgradable' to see them.  
[kali@kali]~  
$ sudo apt install simplescreenrecorder  
Upgrading:  
libavcodec62 libavfilter11 libavformat62 libavutil60 librt1.5-gnutls libswresample6 libswscale9  
Installing:  
simplescreenrecorder  
Installing dependencies:  
libqt5-x11extras5 libstavienc4 simplescreenrecorder-lib
```

- Configure firewall to block malicious traffic.

Configure Firewall to Block Malicious Traffic

What is a Firewall?

A **firewall** is a network security system that monitors and controls incoming and outgoing network traffic based on predefined security rules. It acts as a barrier between trusted internal networks and untrusted external networks, such as the Internet.

The primary purpose of a firewall is to prevent unauthorized access, block malicious traffic, and protect systems from cyber threats.

Objective

The objective of firewall configuration is to:

- Block unauthorized network access.
 - Prevent malicious traffic from reaching systems.
 - Allow only legitimate and necessary communications.
 - Protect sensitive data and services.
 - Reduce the attack surface of the network.
-

How a Firewall Works

A firewall examines network traffic and compares it against configured security rules.

The firewall can:

- Allow traffic.
- Block traffic.
- Log traffic for monitoring and investigation.

Every packet entering or leaving a network is evaluated against the firewall's rules before a decision is made.

Types of Malicious Traffic a Firewall Can Block

1. Unauthorized Connection Attempts

Traffic attempting to access services without permission.

2. Suspicious IP Addresses

Connections originating from known malicious sources.

3. Malware Communication

Traffic used by malware to communicate with external servers.

4. Port Scanning Activity

Repeated connection attempts across multiple ports.

5. Denial-of-Service (DoS) Attempts

Large volumes of traffic intended to overwhelm a system.

6. Unnecessary Services

Traffic targeting services that are not required by the organization.

Firewall Configuration Best Practices

1. Default Deny Principle

A secure firewall configuration follows the principle:

"Deny all traffic by default and allow only what is necessary."

This minimizes unnecessary exposure.

2. Allow Only Required Services

Only essential services should be accessible.

Examples:

- Web traffic (HTTP/HTTPS)
- Email services
- Authorized remote administration services

Unused services should remain blocked.

3. Restrict Access Based on Source

Access can be limited to trusted users, devices, or networks.

Benefits:

- Prevents unauthorized access.

-
- Reduces exposure to external threats.
-

4. Monitor and Log Traffic

Firewalls should record:

- Blocked connections
- Suspicious activities
- Security events

Logs help security teams investigate incidents and identify attack attempts.

5. Block Known Malicious Sources

Organizations may use threat intelligence feeds to identify:

- Malicious IP addresses
- Known attack infrastructure
- Suspicious domains

Traffic associated with these sources can be blocked automatically.

6. Enable Intrusion Prevention Features

Many modern firewalls include additional security capabilities such as:

- Intrusion Detection Systems (IDS)
- Intrusion Prevention Systems (IPS)
- Threat intelligence integration

These features help identify and stop attacks before they reach the target system.

7. Regularly Review Firewall Rules

Firewall rules should be:

- Updated regularly
- Removed if no longer needed

-
- Audited for security weaknesses

Outdated rules may introduce unnecessary risks.

Benefits of Firewall Configuration

- Protects systems from unauthorized access.
 - Reduces exposure to cyber threats.
 - Blocks suspicious and malicious traffic.
 - Improves network security.
 - Supports compliance with security standards.
 - Helps detect potential attacks.
-

Importance in Cybersecurity

Firewalls are one of the first lines of defense in cybersecurity. Proper firewall configuration helps organizations control network communications, reduce attack surfaces, and prevent many common attacks before they reach critical systems.

➤ Disable unused services.

Disable Unused Services

What are Services?

Services are background programs or processes that run automatically on a computer or server to provide specific functions such as networking, printing, file sharing, updates, or remote access.

While many services are necessary for normal system operation, some services may be unnecessary or unused. Keeping unused services enabled can increase security risks and system resource consumption.

What Does "Disable Unused Services" Mean?

Disabling unused services means identifying and turning off services that are not required for the system's intended purpose. This reduces the number of processes running on the system and minimizes potential attack vectors.

It is a fundamental part of **system hardening** and cybersecurity best practices.

Objectives

- Reduce the system's attack surface.
 - Improve overall security.
 - Prevent unauthorized access through unnecessary services.
 - Reduce resource usage (CPU, memory, network).
 - Improve system performance and stability.
-

Why Unused Services Are a Security Risk

Every running service:

- May contain vulnerabilities.
- Can open network ports.
- May provide an entry point for attackers.
- Consumes system resources.

If a service is not needed, leaving it enabled creates unnecessary security exposure.

Example

A file-sharing service running on a system that does not require file sharing could be exploited if a vulnerability exists in that service.

By disabling the service, the potential attack path is removed.

Common Examples of Unused Services

Depending on the system's purpose, services that may not be needed include:

1. FTP Service

- Used for file transfers.
- Often unnecessary if secure alternatives are available.

2. Telnet Service

- Provides remote access.
- Considered insecure because it transmits data without encryption.

3. Print Services

- Can be disabled on systems that do not use printers.

4. Bluetooth Services

- Unnecessary on devices that do not require Bluetooth connectivity.

5. Remote Desktop Services

- Can be disabled if remote administration is not needed.

6. File Sharing Services

- Should be disabled if file sharing is not required.

Process of Disabling Unused Services

Step 1: Identify Running Services

Review all active services running on the system.

Step 2: Determine Business Requirements

Identify which services are essential for system operation.

Step 3: Evaluate Risks

Assess whether a service introduces unnecessary security exposure.

Step 4: Disable Non-Essential Services

Turn off services that are not required.

Step 5: Monitor System Functionality

Ensure the system continues to operate correctly after changes.

Step 6: Review Periodically

Regularly reassess services as system requirements change.

Benefits of Disabling Unused Services

Improved Security

Fewer services mean fewer opportunities for attackers.

Reduced Attack Surface

Attackers have fewer targets available for exploitation.

Better System Performance

Unused services no longer consume CPU, memory, or bandwidth.

Easier Management

A simplified system is easier to monitor and maintain.

Lower Risk of Exploitation

Potential vulnerabilities in disabled services cannot be easily exploited.

Best Practices

- Follow the principle of **least functionality**.
 - Enable only the services required for business operations.
 - Regularly audit running services.
 - Remove or disable outdated and unnecessary software.
 - Document service changes.
 - Monitor logs after configuration changes.
-

Importance in Cybersecurity

Disabling unused services is one of the simplest and most effective security measures. Many cyberattacks target unnecessary services that remain active on systems. By reducing the number of running services, organizations can significantly lower the risk of compromise and improve overall system security.

END..